

NAMA: FAALIH NAZHMI PRASETO

NIM: 103122400065

KELAS: SE0802

MODUL 13

JURNAL

1. Tambahkan 1 toko baru bebas ke koleksi toko, dengan ketentuan:
 - Minimal memiliki 2 produk di dalam array products
 - Isi semua field: name, category, city, address, products Hint:

```
> db.toko.insertOne({
  "name": "Toko Anime",
  "category": "Fashion",
  "city": "Jogja",
  "address": { "street": "Britain", "zipcode": "67676" },
  "products": [
    { "item": "Baju Marin", "price": 250000, "stock": 15 },
    { "item": "Rok Marin", "price": 350000, "stock": 12 }
  ]
})
< {
  acknowledged: true,
  insertedId: ObjectId('6a112ab31be0b8c9e678d734')
}
test>
```

2. Tampilkan semua toko yang di kota Jakarta

```

> db.toko.find({ "city": "Jakarta" })
< {
  _id: ObjectId('6a112a251be0b8c9e678d730'),
  name: 'Toko Berkah',
  category: 'Elektronik',
  city: 'Jakarta',
  address: {
    street: 'Jl. Sudirman',
    zipcode: '10220'
  },
  products: [
    {
      item: 'TV',
      price: 3500000,
      stock: 10
    },
    {
      item: 'Kulkas',
      price: 2800000,
      stock: 5
    }
  ]
}
{
  _id: ObjectId('6a112a251be0b8c9e678d732'),

```

3. Tampilkan toko yang memiliki produk dengan price lebih dari 3.000.000.
Hint:

- Gunakan dot notation: products.price
- Gunakan operator: \$gt

```

> db.toko.find({ "products.price": { $gt: 3000000 } })
< {
  _id: ObjectId('6a112a251be0b8c9e678d730'),
  name: 'Toko Berkah',
  category: 'Elektronik',
  city: 'Jakarta',
  address: {
    street: 'Jl. Sudirman',
    zipcode: '10220'
  },
  products: [
    {
      item: 'TV',
      price: 3500000,
      stock: 10
    },
    {
      item: 'Kulkas',
      price: 2800000,
      stock: 5
    }
  ]
}
{
  _id: ObjectId('6a112a251be0b8c9e678d731'),

```

4. Ubah category Toko Berkah dari Fashion menjadi Aksesoris.
Tambahkan 1 produk baru ke dalam array products milik Toko Maju menggunakan \$push.

Hint:

- Filter berdasarkan name: Toko Maju
- Gunakan operator \$push untuk menambah elemen ke array

```
> db.toko.updateOne(
  { "name": "Toko Berkah" },
  { $set: { "category": "Aksesoris" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.toko.updateOne(
  { "name": "Toko Maju" },
  { $push: { "products": { "item": "Mouse Gaming Fantek", "price": 250000, "stock": 15 } } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

5. Hitung jumlah toko per city, urutkan dari kota yang paling banyak tokonya.
Gunakan:

- \$group dengan \$sum: 1
- \$sort descending (-1)

```
> db.toko.aggregate([
  { $group: { _id: "$city", totalToko: { $sum: 1 } } },
  { $sort: { totalToko: -1 } }
])
< {
  _id: 'Surabaya',
  totalToko: 2
}
{
  _id: 'Jakarta',
  totalToko: 2
}
{
  _id: 'Bandung',
  totalToko: 1
}
```

6. Hitung rata-rata harga produk per category.
Tahapan pipeline:
- \$unwind: '\$products' (bongkar array products dulu)
 - \$group: kelompokkan per category, hitung \$avg dari products.price
 - \$sort: urutkan sesuai kebutuhan

```
> db.toko.aggregate([
  { $unwind: "$products" },
  { $group: { _id: "$category", rataRataHarga: { $avg: "$products.price" } } },
  { $sort: { rataRataHarga: -1 } }
])
< {
  _id: 'Elektronik',
  rataRataHarga: 6114285.714285715
}
{
  _id: 'Aksesoris',
  rataRataHarga: 3150000
}
{
  _id: 'Fashion',
  rataRataHarga: 300000
}
```

7. Tambahkan 1 produk baru ke dalam array products milik Toko Maju menggunakan \$push.

Hint:

- Filter berdasarkan name: Toko Maju
- Gunakan operator \$push untuk menambah elemen ke array

```
> db.toko.updateOne(
  { "name": "Toko Maju" },
  { $push: { "products": { "item": "Keyboard Wireless", "price": 750000, "stock": 8 } } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

8. Tampilkan harga produk tertinggi (\$max) per city.

Hint:

- Unwind products terlebih dahulu
- Gunakan \$max pada products.price

```
> db.toko.aggregate([
  { $unwind: "$products" },
  { $group: { _id: "$city", hargaTertinggi: { $max: "$products.price" } } }
])
< {
  _id: 'Surabaya',
  hargaTertinggi: 12000000
}
{
  _id: 'Jakarta',
  hargaTertinggi: 7700000
}
{
  _id: 'Bandung',
  hargaTertinggi: 8000000
}
```

9. Buat pipeline lengkap dengan 4 tahap:

- \$match — filter hanya toko category Elektronik
- \$unwind — bongkar array products
- \$group — hitung total stock per city

- \$sort — urutkan dari total stock terbesar

```
> db.toko.aggregate([
  { $match: { "category": "Elektronik" } },
  { $unwind: "$products" },
  { $group: { _id: "$city", totalStock: { $sum: "$products.stock" } } },
  { $sort: { totalStock: -1 } }
])
< {
  _id: 'Bandung',
  totalStock: 46
}
{
  _id: 'Surabaya',
  totalStock: 29
}
{
  _id: 'Jakarta',
  totalStock: 18
}
```

10. Hapus toko yang memenuhi dua kondisi sekaligus:

- category adalah Fashion
- city adalah Surabaya

Hint: Gunakan \$and atau tulis kedua kondisi dalam satu objek filter

```
> db.toko.deleteOne({
  $and: [
    { "category": "Fashion" },
    { "city": "Surabaya" }
  ]
})
< {
  acknowledged: true,
  deletedCount: 1
}
test>
```